Text summarization using vectors converts text into numerical representations (vectors) to find important sentences, using methods like **Bag-of-Words (BoW)**, **TF-IDF**, or dense **Word Embeddings (Word2Vec, GloVe)** to capture meaning, then employing algorithms (like similarity matching to a document's topic vector or neural networks) to select key sentences for extractive summaries, effectively turning qualitative text into quantifiable data for ML models.

Key Vectorization Techniques

- **Bag-of-Words (BoW) / Count Vectorizer**: Represents text by word counts, ignoring order, simple but loses context.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**: Weights words by their importance in a document relative to a whole corpus, highlighting unique terms.
- **Word Embeddings (Word2Vec, GloVe, FastText)**: Creates dense, fixed-size vectors capturing semantic relationships and context, offering richer meaning.
- **Transformer-based Embeddings (BERT, etc.)**: Advanced models generating contextual embeddings, capturing nuanced meaning.

Key Vectorization Techniques

- **Bag-of-Words (BoW) / Count Vectorizer**: Represents text by word counts, ignoring order, simple but loses context.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**: Weights words by their importance in a document relative to a whole corpus, highlighting unique terms.
- **Word Embeddings (Word2Vec, GloVe, FastText)**: Creates dense, fixed-size vectors capturing semantic relationships and context, offering richer meaning.
- **Transformer-based Embeddings (BERT, etc.)**: Advanced models generating contextual embeddings, capturing nuanced meaning.

How Vectors Are Used for Summarization (Extractive)

1. **Vectorization**: Convert sentences/documents into vectors using chosen techniques (e.g., TF-IDF or Word Embeddings).
2. **Feature Extraction**: Extract relevant features from these vectors, often including word embeddings, to represent sentence importance.
3. **Sentence Scoring/Selection**:
    1. **Topic Modeling**: Generate a "topic vector" for the whole document and score sentences based on their similarity to it (e.g.,).
    2. **Similarity**: Find sentences similar to key terms or to each other to identify central themes.
    3. **Neural Networks**: Use features from vectors as input to a neural network that classifies sentences as summary-worthy or not.
4. **Summary Generation**: Select the top-scored sentences to form the final extractive summary, often ensuring diversity and avoiding redundancy.

TextRank is an unsupervised, graph-based ranking algorithm used for extractive text summarization and keyword extraction in Python. It is inspired by Google's PageRank algorithm, but instead of ranking web pages, it ranks sentences within a document based on their importance.

How TextRank Works for Text Summarization:

- **Sentence Tokenization:** The input text is first divided into individual sentences.
- **Sentence Representation (Optional but Recommended):** Each sentence is converted into a numerical representation, such as word embeddings or TF-IDF vectors, to enable similarity calculations.
- **Similarity Matrix Construction:** A similarity matrix is created where each cell represents the similarity between two sentences. Common similarity metrics include cosine similarity or Jaccard similarity.
- **Graph Construction:** A graph is built where each sentence is a node (vertex) and edges connect sentences with a similarity score above a certain threshold. The weight of the edges can be the similarity score itself.
- **PageRank Algorithm Application:** The PageRank algorithm is applied to this graph to calculate a "rank" or importance score for each sentence. Sentences that are highly similar to many other important sentences receive higher ranks.
- **Summary Generation:** The top-ranked sentences, based on their importance scores, are selected and combined to form the extractive summary. The number of sentences in the summary can be determined by a fixed number, a percentage of the original text, or a word count limit.

Python Libraries for TextRank and Text Summarization:

Several Python libraries facilitate the implementation of TextRank and other text summarization techniques:

- `gensim` : This library offers a pre-built TextRank implementation for both summarization and keyword extraction.
- `nltk` **(Natural Language Toolkit):** Provides tools for sentence tokenization, stop word removal, and other text preprocessing steps.
- `spaCy` : A powerful NLP library that can be used for tokenization, part-of-speech tagging, and generating word embeddings, which are useful for calculating sentence similarity.
- `networkx` : Useful for creating and manipulating graph structures if you choose to implement TextRank from scratch.
- `pytextrank` : A Python implementation of TextRank specifically designed for keyword and phrase extraction.

Example using `gensim`:

Python

```python
from gensim.summarization import summarize

text = """
The quick brown fox jumps over the lazy dog.
This is a classic sentence used for testing typing skills.
It contains all the letters of the alphabet, making it a pangram.
Pangrams are interesting linguistic curiosities.
"""

# Summarize the text, aiming for a summary that is 50% of the original length
summary = summarize(text, ratio=0.5)
print(summary)

# Summarize the text to a specific word count (e.g., 20 words)
summary_by_word_count = summarize(text, word_count=20)
print(summary_by_word_count)
```